

Chapter 8

Collisions

When this touches that

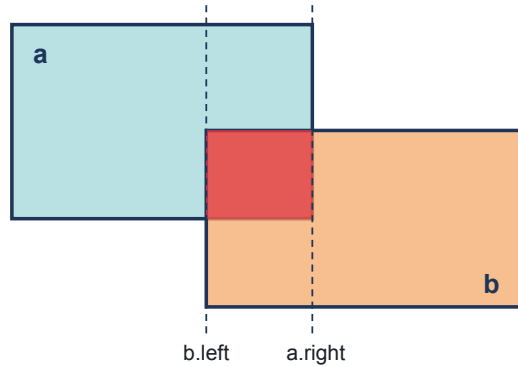


Today

- collisions **by hand**: `Phaser.Geom` shapes and `Intersects` tests
- why rectangles are **unfair** to round things
- **Arcade Physics**: bodies, and `debug` to see them
- fitting a body to its picture
- `collider` or `overlap` ? Groups against groups
- process callbacks, and switching collisions off
- a complete **breakout**

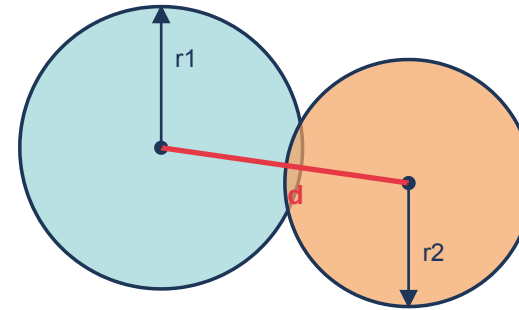
Shapes made of numbers

RectangleToRectangle



they overlap when $a.\text{left} < b.\text{right}$, $a.\text{right} > b.\text{left}$,
 $a.\text{top} < b.\text{bottom}$ and $a.\text{bottom} > b.\text{top}$ - all four

CircleToCircle



they overlap when the distance between the
centres is less than the radii added: $d < r1 + r2$

`Phaser.Geom.Rectangle`, `Circle`, `Triangle` ... - never drawn. `Phaser.Geom.Intersects.RectangleToRectangle(a, b)`,
`CircleToCircle`, `CircleToRectangle`

Which test?

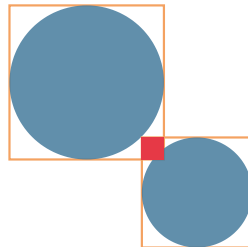
```
if (a instanceof Phaser.GameObjects.Arc && b instanceof Phaser.GameObjects.Arc) {  
    return intersects.CircleToCircle(circleOf(a), circleOf(b));  
}  
if (a instanceof Phaser.GameObjects.Arc) {  
    return intersects.CircleToRectangle(circleOf(a), b.getBounds());  
}  
if (b instanceof Phaser.GameObjects.Arc) {  
    return intersects.CircleToRectangle(circleOf(b), a.getBounds());  
}  
return intersects.RectangleToRectangle(a.getBounds(), b.getBounds());
```

- `getBounds()` - any game object's **bounding box**
- `instanceof` **narrows** the type: `a.radius` is allowed inside the `if`
- every pair, once: `for (j = i + 1 ...)` - 6 shapes, 15 pairs; 100 shapes, 4,950

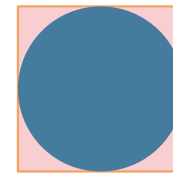
Unfair corners



Why rectangles are unfair to round things



The boxes overlap (red),
so a box test says "hit" -
but the balls are nowhere
near each other.



about a fifth of a circle's box
is empty corner (the red)

Clicking a circle

```
circle.setInteractive({  
    hitArea: new Phaser.Geom.Circle(radius, radius, radius),  
    hitAreaCallback: Phaser.Geom.Circle.Contains,  
    draggable: true,  
    useHandCursor: true,  
});
```

- default hit area: the **rectangle** - a click in the corner picks it up
- hit area coordinates: `(0, 0)` is the **top-left**, so the centre is `(radius, radius)`
- `Contains(shape, x, y)` - on every `Phaser.Geom` class

Try it: Hand-made Collisions

- build, serve, drag the shapes together
- press SPACE: bounding boxes only. Find a "hit" that is not a hit
- click just outside a circle, diagonally. Does it move?

Drag the shapes. Testing: real shapes (SPACE to switch) Touching pairs: 1



Arcade Physics

```
// main.ts
physics: {
  default: "arcade",
  arcade: {
    debug: false,    // true: draw every body
  },
},
```

```
this.physics.add.image(x, y, key);    // a new Image, with a body
this.physics.add.sprite(x, y, key);   // a new Sprite, with a body
this.physics.add.existing(gameObject); // a body for an object you have
```

Bodies - and seeing them

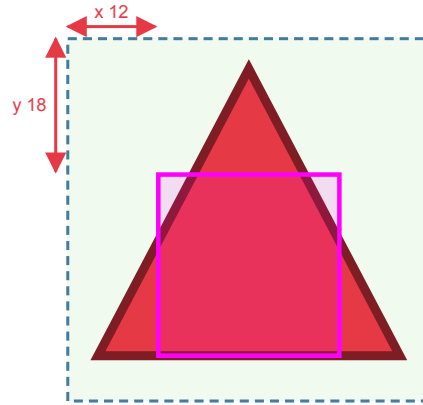
```
export class Player extends Phaser.Physics.Arcade.Image
  constructor(scene: Phaser.Scene, x: number, y: number) {
    super(scene, x, y, PLAYER_KEY);
    scene.add.existing(this);
    scene.physics.add.existing(this);
  }
```

- move with **velocity**: `this.setVelocity(speedX, speedY)`
- pink: dynamic bodies; blue: static



Fitting the body

A body smaller than its picture (enemy.png, drawn 5 times full size)



dashed: the picture, 48 x 48

pink: the body - what the physics engine sees

```
enemy.setSize(24, 24);  
enemy.setOffset(12, 18);
```

setSize: the body's width and height

setOffset: where its top-left corner is,
measured from the picture's top-left corner

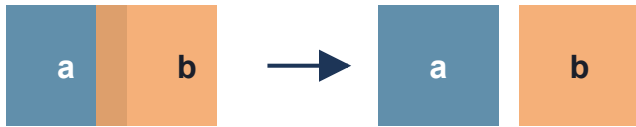
```
star.setCircle(12, 4, 5); // radius, then offset from the top-left
```

Collider or overlap?

this.physics.add.collider(a, b)

1. a moves into b

2. Phaser pushes them apart



separates them, changes their velocities
(bounce), THEN calls your callback

walls, crates, bricks, the paddle

this.physics.add.overlap(a, b)

a moves into b - and nothing is pushed



only calls your callback, every frame
that they still touch

pickups, hazards, trigger areas

Should these two push each other apart?

All the collisions - set up once, in `create()`

```
this.physics.add.collider(this.player, this.crates);
this.physics.add.collider(this.enemies, this.crates);
this.physics.add.collider(this.enemies, this.enemies); // a group against itself

this.physics.add.overlap(this.player, this.stars, (_player, star) => {
  this.collectStar(star as Phaser.Physics.Arcade.Sprite);
});
this.enemyOverlap = this.physics.add.overlap(this.player, this.enemies, () => {
  this.hurtPlayer();
});
```

No collision code in `update()` at all

Groups

```
this.crates = this.physics.add.staticGroup();    // never move
...
this.enemies = this.physics.add.group({          // dynamic
  bounceX: 1,
  bounceY: 1,
  collideWorldBounds: true,
});
...
const crate = this.crates.create(x, y, CRATE_KEY) as Phaser.Physics.Arcade.Sprite;
crate.setScale(CRATE_SCALE);
crate.refreshBody();    // a static body does not follow its game object
```

Callbacks, casts, and switching off

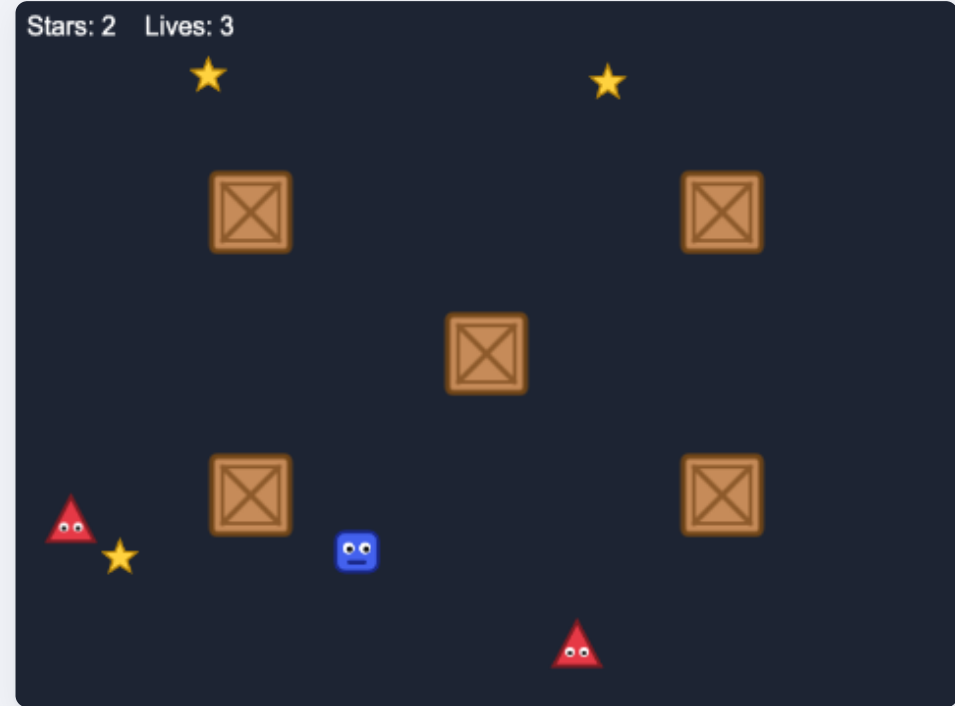
TS2339: Property 'disableBody' does not exist on type 'Body | StaticBody | Tile | GameObjectWithBody' -> star as Phaser.Physics.Arcade.Sprite

```
star.disableBody(true, true);           // body off, hidden
...
star.enableBody(true, spot.x, spot.y, true, true); // back, somewhere else
...
this.enemyOverlap.active = false; // stop checking for a while
...
this.enemyOverlap.active = true; // ...and start again
```

An overlap fires **every frame** they touch

Try it: Collect and Avoid

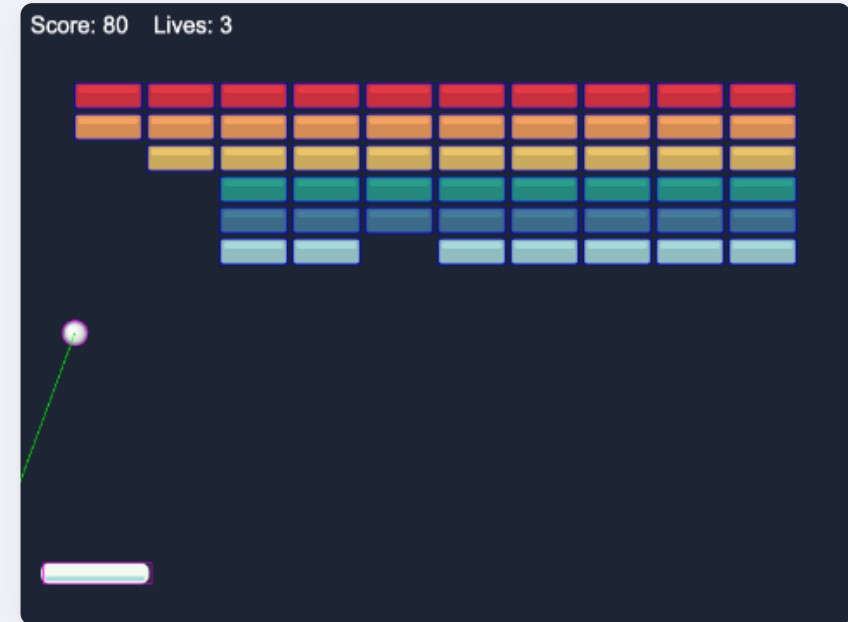
- arrow keys; stars in, triangles out
- set `debug: true` in `main.ts`, rebuild
- replace the stars' overlap with
`this.physics.add.collider(this.player, this.stars).`
What happens?



Try it: Breakout

- paddle: dynamic, **immovable**
- ball: `setCircle`, `setBounce(1)`, world bounds
- bricks: a **static group**, destroyed when hit

```
// no bottom edge: the ball falls out  
this.physics.world.setBoundsCollision(true, true, true, false);
```

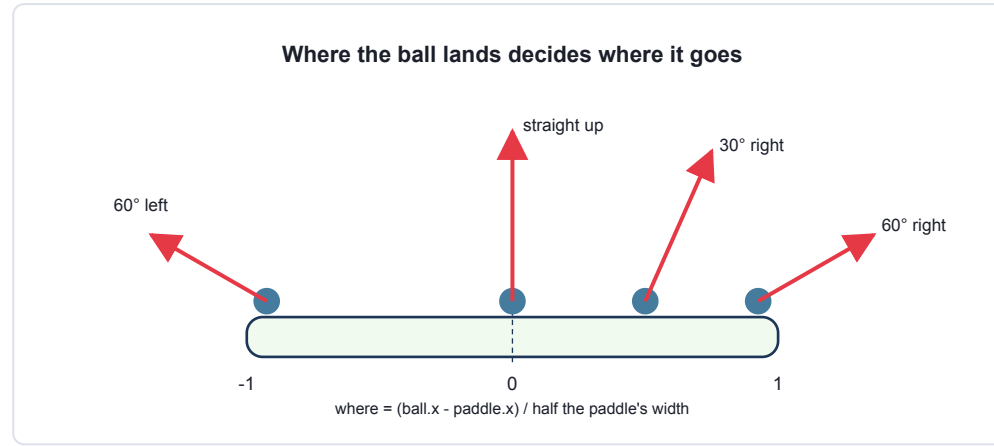


A process callback

```
this.physics.add.collider(  
  this.ball,  
  this.paddle,  
  () => {  
    this.ball.bounceOff(this.paddle);  
    this.sound.play(PADDLE_SOUND);  
  },  
  () => {  
    return this.ball.isFalling(); // false: no collision at all  
  },  
);
```

Runs **first**. Only exactly `false` cancels - forget the `return` and everything collides

Aiming the ball



```
const where = Phaser.Math.Clamp((this.x - paddle.x) / halfWidth, -1, 1);  
this.setDirection(where * MAX_BOUNCE_ANGLE);
```

Summary

- `Phaser.Geom` + `Intersects` ; `getBounds()` ; circles for round things
- physics config; `this.physics.add.image / sprite / existing` ; `debug`
- `setSize` + `setOffset` , `setCircle` - from the **top-left**
- `collider` separates; `overlap` reports. Once, in `create()`
- `group / staticGroup` ; `refreshBody()` ; cast callback objects with `as`
- process callbacks; `collider.active` ; `disableBody` / `enableBody` ; `destroy()`

Challenges

1. **Debug key** - D toggles debug drawing
2. **Hover highlight** - outline the shape under the pointer, fairly
3. **Tough bricks** - the top row takes two hits
4. **Wide paddle** - a falling power-up widens the paddle for 10 s
5. **Glass bricks** - smashed, but no bounce
6. **Fair triangles** - only a real touch of the triangle counts

Next: **Chapter 9 - 2D physics**